WASHDOOR POS & ORDER MANAGEMENT

Supabase Backend Connectivity, Creation & Integration Guide

Project: `washdoor` | Live Supabase Endpoint: `https://eunguuboxwbblioljwmy.supabase.co` | Generated: 10 Nov 2026

1. Backend Connectivity & Environment Setup

1.1 Client Initialization

Supabase connectivity is centralized in `src/lib/supabaseClient.js` and re-exported by compatibility aliases `src/supabaseClient.js` and `src/utils/supabase.js`.

- **Client SDK:** `@supabase/supabase-js` (`^2.118.0`)
- **Environment Variables:** Reads `import.meta.env` in Vite browser builds and falls back to `process.env` in Node.js scripts/tests:
  - `VITE_SUPABASE_URL` — Supabase Project REST/Storage/Auth URL
  - `VITE_SUPABASE_ANON_KEY` / `VITE_SUPABASE_PUBLISHABLE_KEY` — Public anon/publishable key for frontend browser requests
- **Vite Bridge:** `vite.config.ts` maps `REACT_APP_SUPABASE_URL` and `REACT_APP_SUPABASE_ANON_KEY` via `loadEnv`.
- **Health Helper:** `isSupabaseConfigured()` verifies both URL and key are configured before issuing queries.

> **Security Notice:** Never expose `SUPABASE_SERVICE_ROLE_KEY` in `src/` frontend code or `VITE_*` variables. Only the public anon/publishable key is safe for browser bundles.

1.2 Live Connection Status UI

`src/App.jsx` displays a **Supabase Live** status indicator in the top bar whenever `isSupabaseConfigured()` is `true`, linking directly to the Supabase Table Editor.

2. Database Creation & SQL Schema Architecture

2.1 SQL Migration Files (`supabase/`)

File	Responsibility
`supabase/schema.sql`	Core v2.1 PostgreSQL architecture: sequences, `customers`, `customer_works`, 13 relational tables (`washdoor_*`), triggers, `process_wallet_transaction` RPC, views, and RLS policies.
`supabase/customer_works.sql`	Standalone DDL for `customer_works` and `customer_works_summary` view.
`supabase/notification_templates.sql`	`notification_templates` and `notification_logs` tables and indexes.
`supabase/washdoor_orders_master.sql`	Legacy single-table master schema (optional fallback; not deployed on live DB).
`supabase/order_pos_integration_migration.sql`	Incremental migration adding `washdoor_factory_orders`, `washdoor_pickup_delivery_schedules`, `washdoor_customer_addresses`, `washdoor_held_orders`, `washdoor_products` / `washdoor_order_items` DDL parity, `notification_*` tables, `get_next_order_number()`, and extended GST/inspection/photo columns.

2.2 End-to-End Architecture Flow

React UI Modules (`App.jsx`, `POSModule.jsx`, `CustomerManagement.jsx`, `SideEditPanel.jsx`, `EditOrderModal.jsx`, `GarmentInspectionSection.jsx`, `OrderReviewCard.jsx`, `AuditLogModule.jsx`)
↓ *Captures form & cart state*
Service Layer (`customerService.js`, `notificationTemplateService.js`, `columnPreferenceService.js`)
↓ *Normalizes constraints & dispatches API calls*
Supabase Services: PostgREST (`customers`, `customer_works`, `washdoor_*`, `notification_*`) • Storage (`washdoor-garment-photos`) • Auth (`user_metadata`)
↓ *Executes PostgreSQL triggers & generated columns*
Automated Persistence: Computes `balance_due`, syncs `washdoor_payment_ledger` → `advance_paid`, syncs live milestones → `work_status`, protects immutable audit/wallet rows.

2.3 Standardized Business Identifiers

Identifier	Format	Sequence / Function
Order Number	`ORD00001`	`seq_washdoor_order_number` / `generate_order_number()` / `get_next_order_number()`
Customer Code	`CUS00001` / `CUST-00001`	`seq_washdoor_customer_number` / `generate_customer_number()` / `getNextCustomerId()`
Payment Number	`PAY00001`	`seq_washdoor_payment_number` / `generate_payment_number()`
Message / Note / Audit	`MSG00001` / `NOTE00001` / `AUD00001`	`generate_message_number()` / `generate_note_number()` / `generate_audit_number()`
Factory / Held Order	`FAC00001` / `HLD00001`	`generate_factory_order_number()` / `generate_hold_order_number()`

2.4 Triggers, Generated Columns & RPCs

- **Generated Column (`customer_works.balance_due`):** `GENERATED ALWAYS AS (GREATEST(0.00, total_amount - advance_paid)) STORED` — omitted from `INSERT` / `UPDATE` payloads.
- **Payment Auto-Sync (`trg_sync_payment_ledger_to_work`):** Sums `washdoor_payment_ledger.amount` and updates `customer_works.advance_paid` and `payment_status` .
- **Live Status Auto-Sync (`trg_sync_live_action_to_work`):** Updates `customer_works.work_status` when milestones are inserted into `washdoor_live_status_actions` .
- **Immutability (`fn_prevent_audit_tampering`):** Blocks `UPDATE` / `DELETE` on `washdoor_status_history` , `washdoor_wallet_transactions` , and `washdoor_audit_logs` .
- **Atomic Wallet RPC (`process_wallet_transaction`):** Uses row-level `FOR UPDATE` locking on `washdoor_wallets` .

2.5 Live Deployment Inventory (`eunguuboxwbblioljwmy.supabase.co`)

- **20 Live Tables Deployed:** `customers` , `customer_works` , `washdoor_products` , `washdoor_order_items` , `washdoor_garment_cards` , `washdoor_garment_inspections` , `washdoor_inspection_categories` , `washdoor_garment_photos` , `washdoor_status_history` , `washdoor_live_status_actions` , `washdoor_payment_ledger` , `washdoor_payment_gateway_transactions` , `washdoor_wallets` , `washdoor_wallet_transactions` , `washdoor_wallet_customer_monitor` , `washdoor_driver_schedules` , `washdoor_driver_assignments` , `washdoor_customer_messages` , `washdoor_order_notes` , `washdoor_audit_logs` .
- **1 Live Storage Bucket:** `washdoor-garment-photos` (public bucket).
- **Pending SQL Migration:** Execute `supabase/order_pos_integration_migration.sql` once in **Supabase Dashboard → SQL Editor** to create `washdoor_factory_orders` , `washdoor_pickup_delivery_schedules` , `washdoor_customer_addresses` , `washdoor_held_orders` , `notification_templates` , `notification_logs` , `get_next_order_number()` , and extended columns on `customer_works` , `washdoor_garment_cards` , and `washdoor_garment_photos` .

3. Supabase Storage Integration (Garment Photos)

Photos attached in `src/components/GarmentInspectionSection.jsx` are uploaded by `uploadGarmentPhotos()` in `src/services/customerService.js` :

1. Converts base64 `data:` URIs to binary bytes and uploads them to bucket `washdoor-garment-photos` at `{workNumber}/{cardId}/{timestamp}_{idx}_{filename}.{ext}`.
2. Resolves the public URL via `supabase.storage.from('washdoor-garment-photos').getPublicUrl(...)`.
3. Inserts photo metadata (`card_id`, `work_id`, `photo_type`, `storage_path`, `photo_url`, `caption`, `captured_by`, etc.) into `washdoor_garment_photos`.

4. Application Module Integrations

- **Customer CRUD (`CustomerManagement.jsx` + `customerService.js`):** `createCustomer` inserts into `customers` + `customer_works` + `washdoor_customer_addresses`; `getAllCustomers` & `searchCustomers` query `customers` first and deduplicate by `customer_code` / `mobile_number`; `updateCustomer` updates `customers` and `customer_works` registration JSONB without overwriting order `internal_notes`; `deleteCustomer` deletes across all customer tables.
- **Order Creation (`createOrderWithCustomer`):** Resolves/creates customer in `customers`, inserts `customer_works`, syncs `washdoor_order_items`, expands per-piece `washdoor_garment_cards` + `washdoor_garment_inspections` + `washdoor_garment_photos`, upserts `washdoor_factory_orders`, logs initial advance in `washdoor_payment_ledger`, and writes status/note/message/audit entries.
- **Order Read & Update (`getOrders` & `updateOrder`):** Hydrates `washdoor_factory_orders` & `washdoor_pickup_delivery_schedules`; merges existing `internal_notes` JSON safely on partial edits; normalizes statuses & payment modes; syncs items, garment cards/photos, factory orders, schedules, incremental payment ledger entries, status history, and audit logs.
- **Notifications, Preferences & Audit Log:** `notificationTemplateService.js` manages `notification_templates` & `notification_logs`; `columnPreferenceService.js` stores 17-column visibility in `supabase.auth` metadata; `AuditLogModule.jsx` queries `washdoor_audit_logs` & `washdoor_status_history`.

5. Setup Checklist & Live Verification

1. Copy `.env.example` to `.env.local` and configure `VITE_SUPABASE_URL` and `VITE_SUPABASE_ANON_KEY`.
2. Run `supabase/schema.sql` and `supabase/order_pos_integration_migration.sql` in Supabase SQL Editor.
3. Ensure public Storage bucket `washdoor-garment-photos` exists (already live).
4. Verify build via `npm run build` (`tsc --noEmit && vite build` — passed, 502 modules).

Verified Live Records: Customer CRUD cycle (Create/Read/Update/Delete) passed live; sample order `ORD00010` (Lakshmi, 19 garments, On-Call, Ramesh Kumar, ₹50 advance) and `ORD00012` (Anand Kumar, 7 garments, ₹200 advance, 2 PNG photos in Storage + `washdoor_garment_photos`) verified in live Supabase.